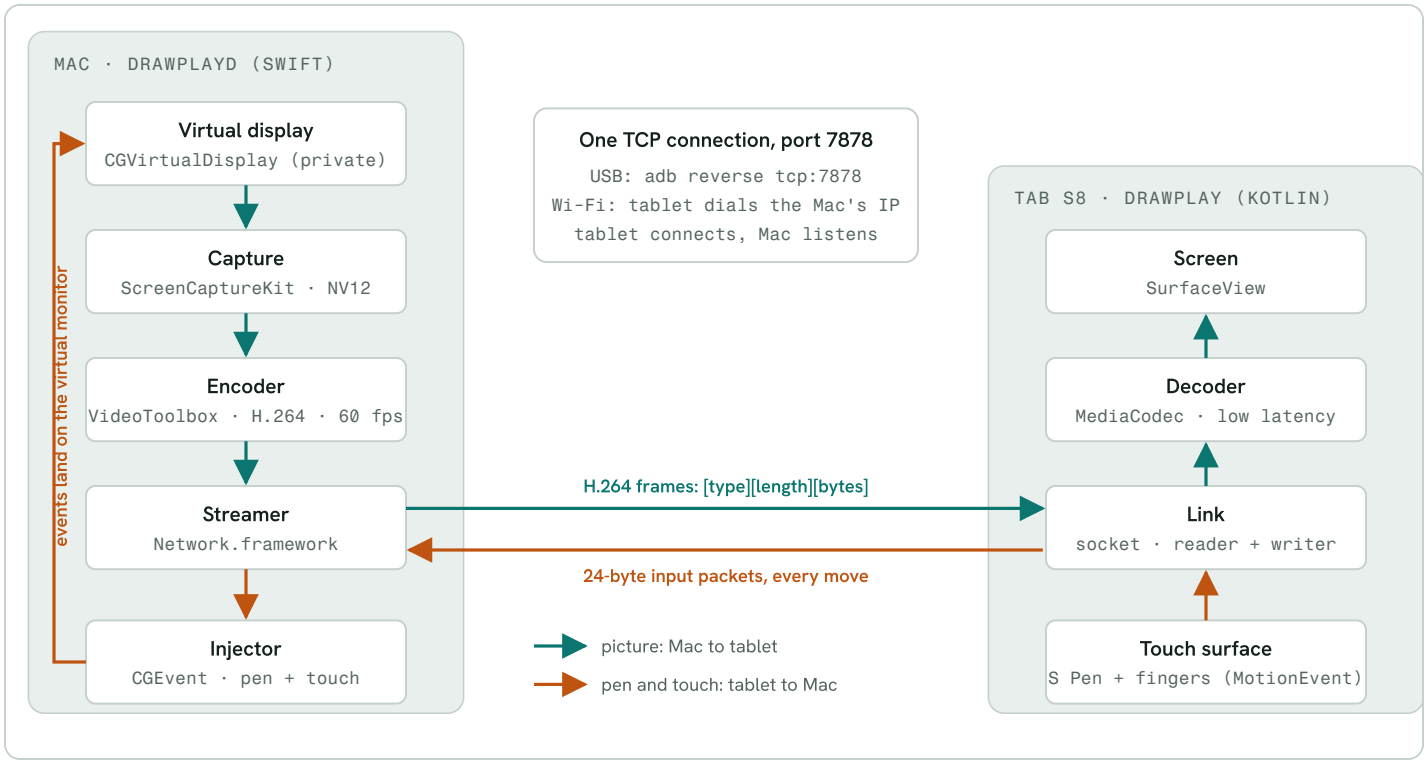
DrawPlay: a Sidecar-style second screen for an Android tablet

The Mac makes a virtual monitor and streams it to a Galaxy Tab S8. The tablet sends pen and finger input back, and the Mac injects it onto that monitor. This page shows how the pieces connect, what travels on the wire, and where macOS's own touch support fits in.

The whole loop



Both directions share one connection. The picture goes down the left stack, across, and up the right stack to the screen. Input goes the other way and is posted by the injector onto the same virtual monitor the tablet is showing, which closes the loop.

What travels on the wire

Tablet to Mac: one 24-byte packet per sample

Mac to tablet: type, length, payload

Little-endian, no framing needed because every packet is the same size. Positions are 0 to 1 across the tablet screen, so the Mac decides the pixel mapping.

H.264 in Annex-B form. The tablet never needs the container format. A config frame carries the SPS/PPS and goes out ahead of every keyframe.

BYTE	FIELD	TYPE	MEANING
0	kind	u8	0 pen tip, 1 eraser end, 2 finger
1	action	u8	0 down, 1 move, 2 up, 3 hover, 4 hover exit
2	pointer	u8	Finger or pen id, so the Mac can track several fingers
3	buttons	u8	Bit 0 pen barrel button (right click), bit 1 secondary
4, 8	x, y	f32 × 2	Position, 0 to 1
12	pressure	f32	0 to 1
16, 20	tiltX, tiltY	f32 × 2	Pen tilt, -1 to 1

TYPE	PAYLOAD	USED FOR
1	SPS + PPS	Starts or restarts the decoder
2	Keyframe	A full picture. Decoding can begin here.
3	Delta frame	Changes since the last frame. Useless without its predecessors.

How input becomes macOS events

INPUT	WHAT THE MAC POSTS	RESULT
Pen near the glass	A tablet-proximity event (pen or eraser), then mouse-moved events marked as tablet points	Drawing apps see a pen hovering
Pen touching	Mouse down, drag and up with the tablet subtype, plus pressure, tilt and button fields	Pressure and tilt reach the app
One finger	Left mouse down, drag, up at the finger's position	Click and drag
Two or more	Pixel scroll events from the movement of the fingers' centre	Two-finger scroll

fingers

Everything is posted at the HID event tap, aimed at the virtual display's bounds. The Mac asks for Accessibility permission for this, and for Screen Recording permission to capture the monitor.

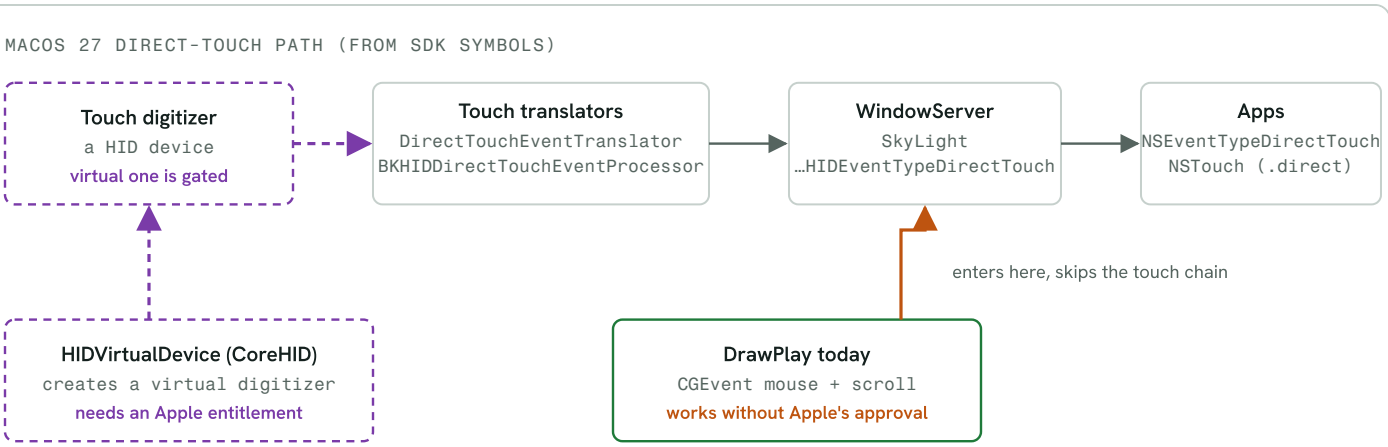
Keeping the picture clean over Wi-Fi

Delta frames depend on every frame before them, so one lost frame smears everything after it until the next keyframe. Both ends handle this the same way: stop showing deltas, ask for a keyframe.

- **Mac:** if more than four sends are still in flight, it drops the frame and forces the encoder to emit a keyframe next. A keyframe is also forced every two seconds.
- **Tablet:** it discards delta frames until a keyframe arrives, and does the same if the decoder had no free input buffer.
- **New connection:** the Mac forces a keyframe and re-encodes the last captured picture, because a still screen produces no new frames to start from.

Where macOS's own touch support fits

macOS 27 appears to contain a direct-touch pipeline, found by reading symbols in the SDK and system libraries. DrawPlay does not use it yet. It posts ordinary mouse and scroll events, which work today without special permission from Apple.



The touch chain is real in the SDK, but nothing here has been seen working. The

background touch daemon (`backboardd`) was not running on the test Mac, so something probably has to start it. The only supported way to feed the chain from user space is the entitlement-gated virtual device.

> **Open question for anyone with the right setup:** does a virtual digitizer, created with the entitlement, make `backboardd` start and deliver real multi-touch to apps? That would turn DrawPlay's two-finger scroll into native pinch, rotate and swipe. Nobody has published an answer that we could find.

Status

PIECE	STATE	NOTE
Virtual monitor	works	Seen on a MacBook Air. Uses a private API that Apple could change.
Video to the tablet	works	Confirmed over Wi-Fi at 1920 × 1200. 2560 × 1600 is now the default.
Pointer control	works	Mouse movement from the tablet is confirmed.
Pen pressure and tilt in apps	untested	Events are sent. Not yet checked in Krita or Photoshop.
Retina mode (`--hidpi`), USB mode	untested	Built, not run.
Pinch, rotate, swipes	not built	Two-finger scroll only.
Smoother strokes, palm rejection	not built	Intermediate pen samples are not sent yet.
Native macOS touch	blocked	Needs the entitlement, or a Mac with code-signing checks relaxed.

Where things live

```
drawplay/
├─ mac/Sources/drawplayd/
│  ├─ main.swift          start-up, wires the pipeline together
│  ├─ VirtualDisplay.swift  creates the second monitor
```

```
│   ├─ VirtualDisplay.swift  creates the second monitor
│   ├─ Capture.swift        ScreenCaptureKit, NV12 frames
│   ├─ Encoder.swift        VideoToolbox H.264, Annex-B output
│   ├─ Streamer.swift       the connection, send queue, back-pressure
│   ├─ Packet.swift         the 24-byte input packet
│   └─ Injector.swift       CGEvent pen, touch and scroll
├─ mac/Sources/VDShim/     header for the private display API
└─ android/app/src/main/kotlin/dev/drawplay/
   └─ MainActivity.kt      Link, Decoder, touch surface, IP panel
```

Needs on the Mac

- Apple Silicon Mac on a recent macOS
- Swift command-line tools (full Xcode is not required)
- Accessibility and Screen Recording permission for the terminal that runs it

Needs on the tablet

- Android 10 or later with a stylus-capable screen
- Same network as the Mac, or a USB cable with debugging on
- Permission to install the APK from outside Play

Related projects

- Touch Up: a user-space macOS driver that turns real USB touchscreens into Mac input. Worth reading for how it handles gestures.
- Weylus: tablet as drawing surface and second screen over a browser. Its pen and multi-touch support is documented for Linux, not macOS.
- PhairPlay: an AirPlay receiver for Android TV. It receives the Mac's picture but carries no input back, which is why this project does not build on it.

Findings about macOS touch come from symbols in the macOS 27.0 SDK and the system library cache, plus one failed entitlement test. They describe what exists, not what works.